ZIPGUARD



Documentation

Comprendre les bombes de décompression pour mieux s'en protéger.

Evan · @Picsou06 • Hamza Karrouchi · @CodeByHaamza

18/10/2026

Sommaire

1 **Comment ça marche** — le mécanisme des zip bombs

2 **Prévention** — onze garde fous et la mesure

3 **Anatomie du dépôt** — un adaptateur par format, un seul jeu de seuils

CHAPITRE 1

Comment fonctionne une bombe de décompression

> *Objectif pédagogique : comprendre le mécanisme pour mieux s'en protéger. Ce dépôt ne fournit aucune bombe prête à l'emploi.*

Le principe

Une **bombe de décompression** (ou *zip bomb*) est une archive volontairement minuscule qui, une fois décompressée, produit une quantité de données énorme, suffisante pour saturer la mémoire vive ou le disque de la machine qui l'ouvre. C'est une attaque par **déni de service** : elle ne vole rien, elle épuise une ressource.

Tout repose sur le **ratio de compression**. Les algorithmes sans perte comme DEFLATE remplacent les répétitions par de courtes références. Un fichier d'un gigaoctet composé du même octet répété se compresse en quelques kilooctets, car l'algorithme n'a qu'à encoder « cet octet, un milliard de fois ».

Les trois familles

Famille	Idée	Détection
Fichier unique très répétitif	Un seul membre énorme et ultra redondant	Ratio de compression anormal
Archive imbriquée	Des archives dans des archives, sur plusieurs niveaux	Inspection récursive jusqu'à `max_depth`, entrées repérées par **signature** et non par extension
Recouvrement d'entrées	Plusieurs entrées pointant vers les mêmes données compressées	Ratio calculé contre la **taille réelle du fichier**, que le recouvrement ne peut pas gonfler

Pourquoi c'est dangereux

Un décompresseur naïf lit l'en-tête, voit un petit fichier, et commence à écrire la sortie sans jamais vérifier combien d'octets il a déjà produits. La mémoire ou le disque se remplit

avant qu'il ne s'arrête. Le service tombe.

La bonne nouvelle : toutes ces familles partagent une signature mesurable **avant** d'écrire quoi que ce soit. C'est exactement ce que lit le scanner de ce dépôt.

Attention au piège de la troisième famille : son but est précisément de fausser la mesure. En faisant pointer vingt entrées vers le même bloc compressé, elle multiplie par vingt la somme des tailles compressées annoncées, donc divise par vingt le ratio apparent. Un scanner qui fait confiance à cette somme la laisse passer. Le seul dénominateur qui ne ment pas est la taille du fichier sur le disque. Voir 02-prevention.md.

CHAPITRE 2

Se protéger des bombes de décompression

La règle d'or : **ne jamais faire confiance aux métadonnées, toujours mesurer en lisant.**

Les onze garde fous

Les six premiers décident du **verdict**, à partir de ce que l'archive raconte et de ce qu'on mesure en la lisant. Les deux suivants protègent le **disque** pendant l'écriture, une fois le verdict rendu, parce qu'un verdict favorable n'est pas un blanc seing. Le neuvième refuse ce qu'on ne peut pas lire, et les deux derniers bornent l'inspection d'une archive distante.

1. **Plafond de taille décompressée totale.** Additionner les tailles annoncées et refuser au delà d'un seuil. ZipGuard : `max_total_bytes`.
2. **Plafond par entrée.** Un seul membre ne doit pas pouvoir dépasser une limite. ZipGuard : `max_entry_bytes`.
3. **Ratio de compression maximal.** Taille décompressée divisée par taille compressée. Un ratio de 1000x est presque toujours une attaque. ZipGuard : `max_ratio`.
4. **Nombre d'entrées maximal.** Des millions de fichiers vides épuisent aussi les ressources. ZipGuard : `max_entries`.
5. **Profondeur d'imbrication maximale.** ZipGuard descend réellement dans les archives filles jusqu'à `max_depth` et rend l'arbre complet des verdicts.
6. **Budget d'inspection des imbriquées.** Borne le nombre total d'archives filles ouvertes. ZipGuard : `max_nested_scans`.
7. **Quota d'octets réellement écrits.** Recompter pendant l'écriture, pas seulement avant. ZipGuard : `max_extract_bytes`.
8. **Marge d'espace libre exigée.** Refuser de commencer, et s'arrêter en cours, si le disque descend sous un plancher. ZipGuard : `min_free_bytes`.
9. **Refus des archives chiffrées.** Ce qu'on ne peut pas lire, on ne peut pas le juger.
10. **Mémoire accordée à une archive imbriquée lue sans disque.** ZipGuard : `max_buffer_bytes`.
11. **Plafond de téléchargement d'une archive distante.** ZipGuard : `max_download_bytes`.

Chaque refus porte en plus de sa phrase un **code stable** (`ratio`, `encrypted`, `nested`…), repris par la sortie JSON et par le rapport SARIF.

Pourquoi mesurer pendant la lecture

Les champs de taille d'un ZIP sont déclaratifs : rien n'oblige une archive malveillante à dire la vérité. Un attaquant peut annoncer une petite taille et fournir un flux qui ne s'arrête

jamais. La seule défense fiable est de **compter les octets réellement produits** et de s'arrêter net au dépassement, bloc par bloc.

Certains formats rendent cette règle obligatoire dès l'inventaire :

Format	Ce qu'il déclare	Ce que ZipGuard fait
`zip` , `7z`	Un manifeste complet	Le lit, puis le vérifie pendant l'écriture
`tar` , `tar.*`	Aucune taille compressée par entrée	Mesure en traversant le flux une fois
`gz` , `bz2` , `xz`	Une taille *modulo 4 Gio* pour gzip	Ignore la déclaration et compte lui même

Pour les formats de la deuxième et de la troisième ligne, lister les entrées impose de décompresser. ZipGuard le fait derrière un compteur qui coupe au seuil, ce qui transforme **l'inventaire lui même en garde fou** : l'archive est refusée en cours de lecture, sans jamais avoir été entièrement déballée.

Cela se voit dans le banc d'essai : le cas `lecture-coupee.tar.gz` annonce 3,8 Mio et le refus tombe après 490 Kio lus.

Reconnaître le format, pas l'extension

Une archive renommée en `notes.txt` reste une archive. ZipGuard reconnaît chaque fichier par sa **signature** (les premiers octets), jamais par son nom. C'est ce qui permet de repérer une bombe imbriquée que son auteur a pris soin de déguiser.

La bombe cachée un niveau plus bas

Une archive externe peut afficher un ratio parfaitement innocent tout en contenant une bombe. Le manifeste externe ne voit qu'un fichier déjà compressé, donc incompressible : ratio 1.0x, verdict rassurant. C'est le cas que tout scanner non récursif laisse passer.

La règle qui en découle : **quand on ne peut pas descendre, on refuse**. Si `max_depth` interdit d'ouvrir une archive fille, son contenu est déclaré invérifiable, et invérifiable vaut refus. Un défenseur ne fait pas confiance à ce qu'il n'a pas pu lire.

Le dernier mot revient à l'écriture

Un verdict favorable n'est pas un blanc seing. Le manifeste a pu mentir, donc l'extraction recompte chaque octet écrit face à `max_extract_bytes` , surveille l'espace libre, et en cas

de dépassement **annule toute l'extraction** avant de remonter l'erreur.

Annuler proprement suppose de ne rien avoir écrit dans la destination. Effacer après coup ne suffit pas : si l'archive contient un `a.txt` et que la destination en avait déjà un, l'effacement emporte le fichier de l'utilisateur avec. ZipGuard écrit donc tout dans un dossier de préparation, placé dans la destination pour rester sur le même disque, et ne déplace l'arborescence qu'une fois la dernière entrée passée. Une extraction refusée ne laisse rien derrière elle, et ne retire rien non plus.

Le verdict lui même doit porter sur ce qui sera extrait. Inspecter un fichier puis le rouvrir pour l'extraire laisse une fenêtre où il peut être remplacé. L'archive est donc recopiée dans un dossier privé, et c'est cette copie, et elle seule, qui est inspectée puis extraite.

Zip Slip et liens symboliques

Un décompresseur doit refuser les noms d'entrée qui sortent du dossier cible. ZipGuard traite les trois pièges classiques avant d'écrire quoi que ce soit :

- le chemin absolu (`/etc/passwd`), que `tar` conserve tel quel ;
- la lettre de lecteur Windows (`C:\Windows\...`) ;
- la remontée par deux points (`../../etc/passwd`).

Le chemin résolu est ensuite vérifié une seconde fois, pour attraper ce que les règles auraient laissé passer.

Les **liens symboliques** sont refusés par défaut, dans `tar` , dans `7z` , comme dans `zip` , où ils se cachent dans les bits de mode des attributs externes. Un lien pointant hors de l'arborescence permet de rediriger une écriture vers n'importe quel fichier du système : c'est une évasion du même genre que le Zip Slip, sous un autre déguisement.

Avec `--allow-links` , un lien est recréé seulement si sa cible **ne fait que descendre** : relative, sans `..` , sans lettre de lecteur. La règle est plus stricte que « reste dans le dossier », et c'est voulu : vérifier qu'une cible reste dedans oblige à suivre les autres liens de l'archive, alors qu'une cible qui ne remonte jamais ne peut pas sortir, quelle que soit la chaîne de liens. Aucune entrée ne peut non plus passer *à travers* un lien de l'archive, et les liens physiques sont toujours refusés.

Deux autres pièges sont arrêtés au même endroit :

- **les entrées en double** : si deux entrées portent le même nom, la seconde écrase la première, et ce qui arrive sur le disque n'est plus ce que l'inspection a vu ;
- **les conflits de forme** : un nom qui sert à la fois de fichier et de dossier, ou un fichier qui prendrait la place d'un dossier déjà présent dans la destination.

Tous ces contrôles ont lieu **pendant une passe de validation préalable**, donc avant la première écriture : une archive piégée ne laisse aucune trace sur le disque.

Les archives chiffrées

Un zip chiffré garde son répertoire central en clair : on y lit des noms et des tailles. C'est précisément le piège. Ces tailles sont déclaratives, et pour les vérifier il faudrait décompresser, donc déchiffrer. Une archive chiffrée qui annonce 1 Mo peut en contenir 10 Go, et une archive imbriquée cachée dedans ne sera jamais reniflée.

ZipGuard refuse donc toute archive dont une seule entrée est chiffrée. Côté zip, c'est le bit 0 des drapeaux de chaque entrée, qui couvre le chiffrement historique comme AES ; côté 7z, `py7zr` dit si l'archive demande un mot de passe, et un 7z à en-têtes chiffrés, dont même la liste est illisible, est refusé dès l'ouverture. À l'extraction, une entrée chiffrée est refusée dans tous les cas, même pour une archive de confiance.

C'était aussi un plantage : avant la version 2.2, ouvrir une entrée chiffrée pour en lire la signature levait une exception que rien ne rattrapait. Pour un outil défensif, planter sur l'entrée hostile est le pire mode de défaillance.

Faire confiance, mais à une empreinte

Certaines archives légitimes ont un ratio qui ferait refuser n'importe quelle bombe : un export de base de données plein de zéros, une image disque vide. Relâcher `max_ratio` pour elles le relâcherait pour tout le monde. La bonne granularité est l'archive elle même, désignée par son **empreinte SHA-256** : `zipguard fingerprint` la calcule, au format de `sha256sum`, et `--trust` lit la liste.

Trois règles encadrent cette confiance :

- elle porte sur les octets exacts : un seul octet changé, et l'archive redevient une inconnue ;
- elle lève le verdict, pas les quotas : l'extraction recompte toujours chaque octet ;
- elle s'applique à la copie privée que l'extraction vient de faire, si bien qu'on ne peut pas faire approuver un fichier et en extraire un autre.

Inspecter sans télécharger

Une archive distante n'a pas besoin d'être posée sur le disque pour être jugée. Un zip range son manifeste à la fin : une requête HTTP par plage sur la fin du fichier, puis une par entrée pour en lire la signature, suffisent. Les archives imbriquées sont lues en mémoire, sous un plafond dédié (`max_buffer_bytes`), et le volume total téléchargé a le sien (`max_download_bytes`), qui devient la raison du refus quand il coupe.

Ce mode refuse plutôt que de deviner : un serveur qui n'annonce pas la taille du fichier prive le ratio de son dénominateur, et un serveur qui recompresse à la volée servirait des octets qui ne sont pas ceux de l'archive.

En production

- Décompresser dans un bac à sable isolé (conteneur en lecture seule, quota disque).
- Journaliser chaque refus avec sa raison : la sortie `--json` est faite pour ça.
- Traiter les archives dans un worker séparé que l'on peut tuer sans affecter le service. C'est exactement ce que fait `zipguard serve`, qui plafonne le dégât d'une archive pathologique à un processus perdu et transforme un délai dépassé en réponse `504`.
- Plafonner la mémoire du worker, pas seulement son temps : un délai ne protège pas d'une allocation géante. `zipguard serve` pose un `RLIMIT_AS` (`--max-memory-mb`) et rend un verdict DANGEREUSE quand il est atteint.
- Enfermer le worker : un analyseur d'archives est du code qui lit des données hostiles, et la question n'est pas de savoir s'il aura un jour une faille, mais ce qu'elle permettra. Sous seccomp, un worker compromis ne peut ni ouvrir le réseau, ni lancer un programme, ni observer un autre processus ; sous ses limites de ressources, il ne peut ni remplir le disque ni la mémoire. `zipguard serve --sandbox strict` refuse d'analyser si l'une de ces couches manque.
- Protéger le service lui même : un délai sur chaque lecture réseau contre les envois au compte goutte, et un nombre borné d'analyses simultanées (`--read-timeout`, `--max-concurrent`).

CHAPITRE 3

Anatomie du dépôt

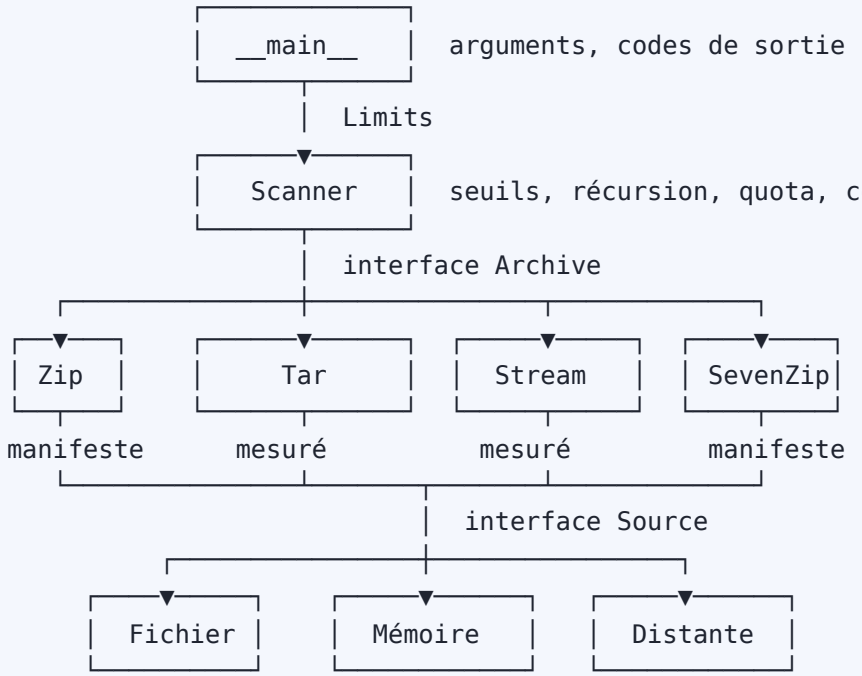
```
zipguard/
├── README.md                 présentation animée et guide de prise en main
├── compose.dev.yaml          orchestration de développement
├── compose.prod.yaml         orchestration de production durcie
├── assets/                   emblèmes et captures, tous générés par script
├── docs/                     cours et explications
│   ├── 01-comment-ca-marche.md
│   ├── 02-prevention.md
│   └── 03-anatomie.md
├── samples/                  archives de démonstration générées localement
├── scripts/
│   ├── build-orbit.py        emblème des contributeurs, avatars embarqués
│   ├── build-terminal.py     capture de la sortie réelle de la commande
│   └── build-pdf.sh          dossier PDF mis en page
└── zipguard.app/             service applicatif (code + Docker)
    ├── Dockerfile
    ├── pyproject.toml
    ├── src/zipguard/
    │   ├── sources.py        origine des octets : fichier local ou mémoire
    │   ├── remote.py         archive distante lue par plages HTTP, sans disque
    │   ├── formats.py        adaptateurs par format et détection par signature
    │   ├── limits.py         seuils immuables, codes de refus et structure du verdi
    │   ├── scanner.py        inspection récursive et extraction comptée
    │   ├── trust.py          empreintes SHA-256 et liste de confiance
    │   ├── report.py         rendu texte et sortie JSON
    │   ├── sarif.py          rapport SARIF pour l'onglet Sécurité de GitHub
    │   ├── service.py        API HTTP locale et worker jetable
    │   ├── sandbox.py        seccomp, limites de ressources et cgroup du worker
    │   ├── bench.py          corpus synthétique et mesure des garde fous
    │   ├── demo.py           générateur de démonstration borné
    │   └── __main__.py       interface en ligne de commande
    └── tests/                couverture du comportement, 261 tests
```

L'idée centrale : le scanner ne connaît aucun format

C'est la décision d'architecture dont tout le reste découle. `scanner.py` ne contient pas une ligne spécifique à ZIP ou à tar : il parle à l'interface `Archive` de `formats.py`, qui expose partout les mêmes trois choses — l'inventaire des entrées, l'ouverture d'une entrée en lecture, et la taille compressée de référence.

Conséquence directe : **ajouter un format ne touche pas aux seuils**, et un `tar.xz` passe exactement les mêmes contrôles qu'un `zip`. Le jour où quelqu'un ajoute `rar`, il écrit un adaptateur, pas une règle de sécurité.

La même idée s'applique une couche plus bas : les adaptateurs ne connaissent pas l'origine des octets. Ils demandent un flux à une `Source` — `FileSource` pour un fichier local, `MemorySource` pour une archive imbriquée lue en mémoire, `RemoteSource` pour une archive servie en HTTP. C'est ce qui permet d'inspecter une archive distante sans la poser sur le disque, avec exactement le même code : seule la source dit si l'inspection a le droit d'écrire un fichier temporaire (`on_disk`) et si son flux sait se déplacer (`seekable`).



Le flux d'une inspection

1. `__main__` lit les arguments et construit un objet `Limits` immuable.
2. `formats.open_archive` reconnaît le format **par signature** et rend l'adaptateur.
3. `Scanner.inspect` inventorie les entrées, derrière un plafond de lecture pour les formats sans manifeste.
4. Les entrées sont reniflées : celles qui sont elles mêmes des archives deviennent des candidates, quel que soit leur nom.
5. Chaque candidate est recopiée dans un tampon borné — un fichier temporaire, ou la mémoire quand la source n'a pas le droit d'écrire —, puis inspectée récursivement avec le même `Scanner` et le même budget.
6. Chaque seuil dépassé ajoute une raison et son code ; un `Verdict` est rendu, avec ses filles. Une archive dont l'empreinte figure dans la liste de confiance s'arrête avant

l'étape 2, déclarée sûre.
7. `report.render` ou `report.render_json` transforme le verdict en sortie.
8. Le code de sortie vaut `0` si l'archive est sûre, `1` sinon, `3` si le fichier est introuvable ou illisible : un script peut distinguer un refus d'une panne.

Le flux d'une extraction

L'extraction ne fait pas confiance au verdict qui vient d'être rendu.

1. L'archive est recopiée dans un dossier de travail privé, créé dans la destination. C'est cette copie qui est inspectée puis extraite. Un verdict défavorable arrête tout.
2. Une passe de validation préalable résout **tous** les chemins, refuse les liens (ou, avec `--allow-links`, ceux qui remontent), les doublons et les conflits avec ce qui existe déjà, avant la première écriture.
3. L'espace libre est vérifié, et le sera encore périodiquement pendant l'écriture.
4. Chaque entrée est écrite par blocs dans un dossier de préparation, chaque bloc recompté face à `max_entry_bytes` et à `max_extract_bytes`.
5. Seulement si tout est passé, l'arborescence préparée est déplacée dans la destination.
6. Dans tous les cas, le dossier de travail est supprimé. Une extraction ratée ne laisse donc rien derrière elle, et ne touche à aucun fichier préexistant.

La frontière de sécurité

Le module `demo.py` plafonne la taille à 100 Mo et ne produit qu'une seule entrée, non imbriquée, qui se décompresse exactement à la taille annoncée, en `zip` comme en `tar.gz`. Le corpus de `bench.py` suit la même règle : aucune archive au delà d'un mégaoctet compressée, aucune imbrication au delà d'un niveau.

Toute la logique offensive est donc absente du dépôt : il n'y a que de la mesure et de la défense.

Les limites connues, écrites noir sur blanc

Un outil défensif qui cache ses angles morts n'est pas défendable. Ceux de ZipGuard :

- **7z** passe par un dossier temporaire, ou un tampon mémoire pour une source sans disque, pour lire une entrée, parce que `py7zr` ne sait pas rendre un flux borné. La taille annoncée est vérifiée avant, la taille réelle après : un manifeste menteur est détecté une fois l'entrée décompressée, pas avant.
- **Lister un `tar` compressé impose de traverser tout le flux.** C'est inhérent au format, pas un choix : le plafond de lecture borne le coût, il ne le supprime pas.

- **Les archives chiffrées** ne sont pas inspectables : elles sont détectées et refusées, pas lues.
- **Le filtre seccomp est une liste de refus**, pas une liste blanche. Il ferme ce qu'un analyseur compromis chercherait en premier (réseau, exécution, autres processus, noyau), pas tout. Il n'existe que sous Linux x86_64 et aarch64 ; ailleurs, `--sandbox strict` refuse d'analyser plutôt que de faire semblant.
- **Le cgroup suppose une délégation** du cgroup v2 à l'utilisateur du service. Sans elle, les limites de ressources par processus restent le seul plafond.
- **Une archive de confiance distante** est téléchargée en entier pour calculer son empreinte, sous le même plafond de téléchargement.